Degree Planner: Complete Step-by-Step Build Guide

A beginner-friendly build plan for a degree-planning web app (React + TypeScript, Spring Boot, PostgreSQL, Jsoup), starting with George Mason University.

How to use this guide

Follow the parts in order. Each part ends with a **checkpoint**: a concrete thing that must work before you move on. Do not skip checkpoints. The most common reason first-time projects fail is building five things at once and having none of them work.

The guide is split into:

- **Part 0**: how the pieces fit together (read once)
- **Part 1**: install tools
- **Part 2**: create the project skeleton
- **Part 3**: design the database (with the reasoning)
- **Part 4**: hand-seed one major and build the backend API
- **Part 5**: build the frontend
- **Part 6**: the scraper
- **Part 7**: accounts and login
- **Part 8**: automatic plan generation
- **Part 9**: testing
- **Part 10**: deployment
- **Part 11**: timeline and team workflow

One important change from your original roadmap: **we build the app with hand-entered data for one major first, and write the scraper after.** A scraper is the riskiest, messiest part. If you build it first, you will not know what shape your data needs to be. If you build the app first, the scraper's job becomes simple: fill tables you already understand.

Part 0: How the pieces fit together

There are three programs running when you develop:

1. **PostgreSQL** (database): stores courses, majors, students. It runs in Docker so everyone on your team has an identical setup.
2. **Spring Boot** (backend, port 8080): a Java program that reads and writes the database and exposes a **REST API**, meaning URLs like `GET /api/courses?search=CS` that return JSON.
3. **React app** (frontend, port 5173): runs in the browser, calls the backend's URLs, and draws the pages.

The flow for any feature is always the same:

Browser (React) → HTTP request → Controller → Service → Repository → PostgreSQL, then the data travels back the same way as JSON.

- **Controller**: receives the HTTP request, returns JSON. No logic.
- **Service**: the actual logic (calculating degree progress, generating plans).
- **Repository**: talks to the database.
- **Entity**: a Java class that maps to a database table.
- **DTO**: a small Java class shaped exactly like the JSON you want to send to the frontend (so you never expose raw database entities).

The scraper is a separate Java job inside the backend that runs occasionally (not on every request), reads GMU's catalog pages, and inserts rows into the database.

Part 1: Install your tools

Every teammate installs these:

1. **Git**, and a GitHub account.
2. **JDK 21** (Eclipse Temurin is a good free build). Check with `java -version`.
3. **Node.js (current LTS)**. Check with `node -v` and `npm -v`.
4. **Docker Desktop**. Check with `docker --version`.
5. **IntelliJ IDEA Community Edition** for the backend (it has the best Java and Spring support).
6. **VS Code** for the frontend, with the ESLint and Prettier extensions.
7. **Postman** or the VS Code REST Client extension, for testing the API by hand.
8. A database viewer: **DBeaver** (free) or pgAdmin, to look inside PostgreSQL.

Checkpoint: every teammate can run all four version commands (`git`, `java`, `node`, `docker`) in a terminal and see a version number.

Part 2: Create the project skeleton

We use a **monorepo**: one GitHub repo containing both the frontend and backend. It is simpler for a small team, since one pull request can change both sides of a feature.

Step 2.1: Clone and set up the repo folders

```
git clone https://github.com/YOUR-ORG/YOUR-REPO.git
cd YOUR-REPO
mkdir backend frontend docs
```

Target structure:

```
YOUR-REPO/
  backend/          Spring Boot project
  frontend/         React project
  docs/             notes, diagrams, decisions
  docker-compose.yml
  .gitignore
  README.md
  LICENSE
```

Step 2.2: Add a .gitignore

Create `.gitignore` in the repo root:

```
# Java / Maven
backend/target/
*.class

# Node
frontend/node_modules/
frontend/dist/

# IDEs
.idea/
.vscode/
*.iml

# Secrets and local config
.env
*.local

# Scraper cache
backend/scraper-cache/
```

Never commit passwords or secrets. Anything secret goes in a `.env` file that is ignored by Git.

Step 2.3: Start PostgreSQL with Docker

Create `docker-compose.yml` in the repo root:

```
services:
  db:
    image: postgres:16
    container_name: degreeplanner-db
    environment:
      POSTGRES_DB: degreeplanner
      POSTGRES_USER: planner
      POSTGRES_PASSWORD: planner_dev_pw
    ports:
      - '5432:5432'
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Start it:

```
docker compose up -d
```

The `volumes` section keeps your data when you stop the container. The password is fine for local development only; production will use a different one (Part 10).

Checkpoint: open DBeaver, create a connection to `localhost:5432`, database `degreeplanner`, user `planner`, and connect successfully.

Step 2.4: Generate the Spring Boot project

1. Go to **start.spring.io**.
2. Choose: **Project:** Maven, **Language:** Java, **Spring Boot:** the latest stable release (not a SNAPSHOT or milestone), **Java:** 21.
3. Group: `com.degreeplanner`, Artifact: `backend`.
4. Add these dependencies:
   - **Spring Web** (REST controllers)
   - **Spring Data JPA** (database access)
   - **PostgreSQL Driver**
   - **Flyway Migration** (database version control)
   - **Validation**
   - **Spring Boot Actuator** (health check)
5. Click Generate, unzip, and copy the **contents** into your `backend/` folder so that `backend/pom.xml` exists.
6. Open the `backend/` folder in IntelliJ and let Maven download dependencies.

We add Spring Security (Part 7) and Jsoup (Part 6) later, when we need them. Adding things only when needed keeps errors easier to understand.

Step 2.5: Configure the database connection

Rename `backend/src/main/resources/application.properties` to `application.yml` and use:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/degreeplanner
    username: planner
    password: planner_dev_pw
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
  flyway:
    enabled: true

server:
  port: 8080
```

The key setting is `ddl-auto: validate`. It means Hibernate will **never create or change tables itself**; it only checks that your Java classes match the database. Tables are created only by Flyway migration files (explained in Part 3). This prevents the classic disaster of Hibernate silently altering your schema.

Step 2.6: Create a health endpoint

Create `backend/src/main/java/com/degreeplanner/backend/web/HealthController.java`:

```
package com.degreeplanner.backend.web;

import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class HealthController {

    @GetMapping("/health")
    public Map<String, String> health() {
        return Map.of("status", "ok");
    }
}
```

Before this will start you need at least one Flyway migration, so create `backend/src/main/resources/db/migration/V1__init.sql` containing just `SELECT 1;` for now (we replace it in Part 3). Run the app from IntelliJ (the `BackendApplication` class) and open `http://localhost:8080/api/health` in your browser. You should see `{"status":"ok"}`.

Step 2.7: Allow the frontend to call the backend (CORS)

Browsers block a page on port 5173 from calling port 8080 unless the server allows it. Create `backend/src/main/java/com/degreeplanner/backend/config/WebConfig.java`:

```
package com.degreeplanner.backend.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("http://localhost:5173")
                .allowedMethods("GET", "POST", "PUT", "DELETE");
    }
}
```

Step 2.8: Create the React + TypeScript frontend

From the repo root:

```
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
npm install axios @tanstack/react-query react-router-dom
npm run dev
```

Vite is the modern standard tool for starting React projects (it is much faster than the older Create React App, which is no longer recommended). The extra packages: **axios** (HTTP calls), **TanStack Query** (fetching and caching server data), **React Router** (multiple pages).

Open `http://localhost:5173`.

Step 2.9: Connect frontend to backend

Replace `frontend/src/App.tsx` with:

```
import { useEffect, useState } from 'react';
import axios from 'axios';

export default function App() {
  const [status, setStatus] = useState('loading...');

  useEffect(() => {
    axios
      .get('http://localhost:8080/api/health')
      .then((res) => setStatus(res.data.status))
      .catch(() => setStatus('backend unreachable'));
  }, []);

  return <h1>Backend status: {status}</h1>;
}
```

Checkpoint (this is your Phase 1 deliverable): the browser shows `Backend status: ok`. Commit and push.

Step 2.10: Team workflow on GitHub

Do this now, before anyone writes feature code:

- Protect the `main` branch (Settings → Branches): require a pull request and one approval before merging.
- Branch naming: `feature/course-search`, `fix/prereq-parser`.
- Small pull requests (under about 300 lines changed). Big ones never get reviewed properly.
- Commit messages in the imperative: `Add course entity`, not `stuff`.
- Add a basic GitHub Actions workflow later (Part 9).
- Put setup instructions in the README as you go. If a teammate cannot start the project from the README alone, the README is wrong.

Part 3: Design the database

This is the most important design decision in the project. Get it right and everything later is easy; get it wrong and you rewrite things in Phase 5.

The principles behind the design

1. **One table per real-world thing**: universities, courses, programs (majors), users.
2. **Never store lists in a single column.** A major has many courses, and a course belongs to many majors. That relationship gets its own table (a *join table*).
3. **Degree requirements are a tree.** Real requirements look like this:

```
BS Computer Science (120 credits)
  ├─ ALL of: Core courses        (CS 110, CS 211, CS 222, ...)
  ├─ CHOOSE 2 from: Math options (MATH 113, MATH 114, STAT 346)
  ├─ CHOOSE 12 credits from: CS electives at 300+ level
  └─ General education
        ├─ ALL of: Written communication
        └─ CHOOSE 1 from: Arts
```

Each box is a **requirement group** with a rule (ALL, CHOOSE N courses, or MIN credits). Groups can contain other groups. That is the recursive design in the schema below. 4. **Keep raw text and parsed data.** A course's prerequisite is free text in the catalog. We store the original text *and* a parsed JSON version, so a parsing mistake never destroys information and we can improve the parser later. 5. **Plan for more universities now** with a `university_id` on top-level tables. It costs almost nothing now and saves a rewrite later. 6. **Use migrations, not manual changes.** Every database change is a numbered SQL file in `db/migration/`. Flyway runs them in order and records which ran. Teammates get your changes by pulling the code. **Never edit a migration file after it has been merged**; add a new one instead.

The tables

Replace `backend/src/main/resources/db/migration/V1__init.sql` with the following (this is safe to replace now because nothing has been merged using it yet):

```
-- Universities (GMU first, others later)
CREATE TABLE university (
    id          BIGSERIAL PRIMARY KEY,
    name        VARCHAR(200) NOT NULL,
    slug        VARCHAR(50)  NOT NULL UNIQUE   -- e.g. 'gmu'
```

```
);

-- Courses
CREATE TABLE course (
    id             BIGSERIAL PRIMARY KEY,
    university_id  BIGINT       NOT NULL REFERENCES university(id),
    subject        VARCHAR(10)  NOT NULL,       -- 'CS'
    number         VARCHAR(10)  NOT NULL,       -- '211' (text, since some
are '101L')
    title          VARCHAR(300) NOT NULL,
    credits_min    NUMERIC(3,1) NOT NULL,
    credits_max    NUMERIC(3,1) NOT NULL,       -- same as min unless
variable credit
    description    TEXT,
    prereq_text    TEXT,                        -- raw catalog text, never
modified
    coreq_text     TEXT,
    prereq_tree    JSONB,                       -- parsed logic, see below
    parse_status   VARCHAR(20) NOT NULL DEFAULT 'NONE',  -- NONE, PARSED,
NEEDS_REVIEW
    source_url     VARCHAR(500),
    scraped_at     TIMESTAMP,
    UNIQUE (university_id, subject, number)
);
CREATE INDEX idx_course_search ON course (university_id, subject, number);

-- Which terms a course is usually offered
CREATE TABLE course_offering (
    course_id  BIGINT      NOT NULL REFERENCES course(id) ON DELETE CASCADE,
    term_type  VARCHAR(10) NOT NULL CHECK (term_type IN
('FALL','SPRING','SUMMER')),
    PRIMARY KEY (course_id, term_type)
);

-- A major (or minor) for a particular catalog year
CREATE TABLE program (
    id                 BIGSERIAL PRIMARY KEY,
    university_id      BIGINT       NOT NULL REFERENCES university(id),
    name               VARCHAR(200) NOT NULL,      -- 'Computer Science'
    degree_type        VARCHAR(20)  NOT NULL,      -- 'BS', 'BA'
    catalog_year       VARCHAR(9)   NOT NULL,      -- '2025-2026'
    total_credits      INT          NOT NULL,
    UNIQUE (university_id, name, degree_type, catalog_year)
```

```
);

-- Requirement tree: each row is a box in the tree
CREATE TABLE requirement_group (
    id           BIGSERIAL PRIMARY KEY,
    program_id   BIGINT       NOT NULL REFERENCES program(id) ON DELETE
CASCADE,
    parent_id    BIGINT       REFERENCES requirement_group(id) ON DELETE
CASCADE,  -- NULL = top level
    name         VARCHAR(200) NOT NULL,
    rule_type    VARCHAR(20)  NOT NULL CHECK (rule_type IN
('ALL','CHOOSE_N','MIN_CREDITS')),
    min_courses  INT,           -- used by CHOOSE_N
    min_credits  NUMERIC(4,1),  -- used by MIN_CREDITS
    sort_order   INT NOT NULL DEFAULT 0
);

-- Which courses can satisfy a group
CREATE TABLE requirement_group_course (
    group_id   BIGINT NOT NULL REFERENCES requirement_group(id) ON DELETE
CASCADE,
    course_id  BIGINT NOT NULL REFERENCES course(id),
    PRIMARY KEY (group_id, course_id)
);

-- Users
CREATE TABLE app_user (
    id             BIGSERIAL PRIMARY KEY,
    email          VARCHAR(255) NOT NULL UNIQUE,
    password_hash  VARCHAR(100) NOT NULL,
    university_id  BIGINT REFERENCES university(id),
    program_id     BIGINT REFERENCES program(id),
    start_year     INT,
    start_term     VARCHAR(10),
    created_at     TIMESTAMP NOT NULL DEFAULT now()
);

-- A student's courses: completed, in progress, or planned
CREATE TABLE student_course (
    id             BIGSERIAL PRIMARY KEY,
    user_id        BIGINT      NOT NULL REFERENCES app_user(id) ON DELETE
CASCADE,
    course_id      BIGINT      NOT NULL REFERENCES course(id),
```

```
    status         VARCHAR(15) NOT NULL CHECK (status IN
('COMPLETED','IN_PROGRESS','PLANNED')),
    planned_year   INT,           -- calendar year, e.g. 2027
    planned_term   VARCHAR(10),   -- 'FALL', 'SPRING', 'SUMMER'
    UNIQUE (user_id, course_id)
);

INSERT INTO university (name, slug) VALUES ('George Mason University',
'gmu');
```

Why each choice was made

- **`course.number` is text**, because some course numbers contain letters.
- **`credits_min` / `credits_max`** exist because some courses are variable-credit (1 to 3 credits).
- **`prereq_text` plus `prereq_tree`**: the tree is JSONB so PostgreSQL stores it natively and you do not need extra tables for it. Example for "CS 211 and (MATH 113 or MATH 108)":

```
{
  "type": "AND",
  "children": [
    { "type": "COURSE", "code": "CS 211" },
    {
      "type": "OR",
      "children": [
        { "type": "COURSE", "code": "MATH 113" },
        { "type": "COURSE", "code": "MATH 108" }
      ]
    }
  ]
}
```

- **`parse_status`**: when the parser cannot understand something ("permission of instructor", "placement exam"), it marks the course `NEEDS_REVIEW` so a human can fix it. Treat those courses as having no enforced prerequisite in the planner, but show the raw text to the student.
- **`requirement_group.parent_id`** makes the tree. A top-level group has `NULL`; a child points at its parent.

- **`rule_type`**: `ALL` means every listed course is required; `CHOOSE_N` means pick `min_courses` of them; `MIN_CREDITS` means collect `min_credits` credits from them. A group with children but no direct courses (like "General Education") is satisfied when its children are.
- **`student_course.status` with planned year and term** lets one table hold both degree progress and the semester plan. Completed courses have no planned semester; planned ones do.
- **`course_offering`** is separate because the catalog does not reliably tell you when courses run. You will fill it from the schedule of classes or by hand.
- **Catalog year** lives on `program`, so a student who entered in 2024 follows the 2024 requirements even after the rules change.

Handling catalog years honestly

Courses themselves also change slightly each year. For your first version, store one version of each course (the most recent catalog) and tie *requirements* to the catalog year. Note that limitation in the README; versioning course text by year is a good future improvement.

Checkpoint: restart the backend. In DBeaver you should see all the tables, plus a `flyway_schema_history` table recording that V1 ran, and one row in `university`.

Part 4: Hand-seed one major and build the backend API

Goal: get a complete working backend for **one major (Computer Science)** with data you type in yourself. This proves your schema works.

Step 4.1: Seed sample data

Create `V2__seed_sample_data.sql` in `db/migration/`. Start small: 10 to 15 courses, one program, a few groups.

```
INSERT INTO course (university_id, subject, number, title, credits_min,
credits_max, prereq_text, prereq_tree, parse_status)
VALUES
 (1,'CS','110','Essentials of Computer Science',3,3,NULL,NULL,'NONE'),
 (1,'CS','211','Object-Oriented Programming',3,3,'CS 110',
   '{"type":"COURSE","code":"CS 110"}','PARSED'),
 (1,'MATH','113','Analytic Geometry and Calculus I',4,4,NULL,NULL,'NONE'),
 (1,'MATH','114','Analytic Geometry and Calculus II',4,4,'MATH 113',
   '{"type":"COURSE","code":"MATH 113"}','PARSED');

INSERT INTO program (university_id, name, degree_type, catalog_year,
total_credits)
VALUES (1,'Computer Science','BS','2025-2026',120);

INSERT INTO requirement_group (program_id, parent_id, name, rule_type,
sort_order)
VALUES (1, NULL, 'Major Core', 'ALL', 1);

INSERT INTO requirement_group_course (group_id, course_id)
SELECT 1, id FROM course WHERE subject='CS' AND number IN ('110','211');
```

Check course codes and requirements against the real GMU catalog for your chosen major. This is also a good time to learn how the real requirements are written, because it will shape everything else. The numbers above are placeholders.

Step 4.2: Backend package structure

```
com.degreeplanner.backend/
  web/        controllers (HTTP endpoints)
  service/    business logic
  repository/ database access
  model/      entities (map to tables)
  dto/        request/response shapes
  config/     CORS, security
  scraper/    Part 6
  planner/    Part 8
```

Step 4.3: Write your first entity, repository, service, controller

Entity (`model/Course.java`). Add the other columns the same way; IntelliJ can generate getters and setters (Alt+Insert), or use Lombok if your team prefers.

```
@Entity
@Table(name = "course")
public class Course {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String subject;
    private String number;
    private String title;

    @Column(name = "credits_min")
    private BigDecimal creditsMin;

    @Column(name = "prereq_text")
    private String prereqText;

    @JdbcTypeCode(SqlTypes.JSON)
    @Column(name = "prereq_tree", columnDefinition = "jsonb")
    private JsonNode prereqTree;

    // ... other columns, getters, setters
}
```

Repository (`repository/CourseRepository.java`):

```
public interface CourseRepository extends JpaRepository<Course, Long> {
    @Query("select c from Course c where lower(c.subject) like
lower(concat(:q, '%')) " +
           "or lower(c.title) like lower(concat('%', :q, '%'))")
    List<Course> search(@Param("q") String q);
}
```

Controller (`web/CourseController.java`) with `GET /api/courses?q=cs` and `GET /api/courses/{id}` , calling a `CourseService` , which calls the repository. Return **DTOs** (a record like `CourseDto(Long id, String code, String title, BigDecimal credits, String prereqText)`) rather than the entity.

Checkpoint: `http://localhost:8080/api/courses?q=CS` in your browser returns your seeded courses as JSON.

Step 4.4: Build the rest of the API (in this order)

1. `GET /api/programs` (list majors) and `GET /api/programs/{id}/requirements` (the requirement tree as nested JSON).
2. `GET/POST/DELETE /api/me/courses` (mark courses completed, planned, or remove). For now, use a hardcoded test user id; real login comes in Part 7.
3. `GET /api/me/progress` (degree progress, below).
4. `GET /api/courses/{id}/eligibility` (can this student take this course now?).

Step 4.5: Prerequisite checking

Prerequisite evaluation is a small recursive function over the JSON tree:

```
satisfied(node, completedCodes):
   COURSE -> completedCodes contains node.code
   AND    -> every child satisfied
   OR     -> at least one child satisfied
   null   -> true  (no prerequisite, or needs manual review)
```

Write this as a plain Java class with no database access (`PrerequisiteEvaluator`). Pure functions like this are easy to unit test, and you will reuse it in the plan generator.

Step 4.6: Degree progress calculation

For each requirement group, compute a status from the student's completed courses. Recursive logic:

```
evaluate(group, completed):
   matched   = courses in group that the student completed
   children  = evaluate(each child group)

   ALL         -> done when every course in group is completed AND every
child is done
   CHOOSE_N    -> done when matched.count >= min_courses
   MIN_CREDITS -> done when sum(credits of matched) >= min_credits

   return { group, done, creditsEarned, remaining courses/credits, children }
```

Return this whole tree to the frontend. The dashboard in Part 5 just draws it.

The double-counting problem: one course might appear in two groups (for example a math course counting for both the major and general education). Decide your policy and write it down. Simplest first version: allow a course to count in every group where it appears, and note that real GMU rules may restrict overlap. You can refine it later.

Checkpoint: you can mark `CS 110` as completed through Postman and see the Major Core group go from 0 of 2 to 1 of 2.

Part 5: Build the frontend

Build pages in this order, each one working against the real API before starting the next.

Step 5.1: Folder structure

```
frontend/src/
  api/          axios client + one file per resource (courses.ts,
programs.ts)
  components/   reusable pieces (CourseCard, ProgressBar, ...)
  pages/        one file per page (Dashboard, Catalog, Roadmap, Login)
  hooks/        data hooks built on TanStack Query
  types/        TypeScript types matching your DTOs
  App.tsx       routes
```

Step 5.2: Set up the foundations

- **Axios client** (`api/client.ts`): one instance with `baseURL` from an env variable (`import.meta.env.VITE_API_URL`), so changing the backend address never means editing code.
- **TanStack Query** provider wrapped around the app, and **React Router** with routes for `/`, `/catalog`, `/roadmap`, `/login`.
- **TypeScript types** that mirror your DTOs, for example `type Course = { id: number; code: string; title: string; credits: number; prereqText?: string }`.
- A UI approach: plain CSS is fine; Tailwind CSS or a component library such as Material UI will make you faster.

Step 5.3: Pages in order

1. **Catalog page**: a search box plus a list of courses. Use `useQuery` with the search text as the key. Debounce typing so you do not fire a request on every keystroke. Clicking a course opens a details panel showing the description, prerequisite text, and whether the student is eligible.
2. **Profile / setup page**: pick a major, and a checklist to mark completed courses.
3. **Dashboard**: total credits earned versus required, and the requirement tree drawn as expandable sections with progress bars. Color-code groups: done, in progress, not started.
4. **Roadmap page**: a grid with one column per semester (Fall Year 1, Spring Year 1, ...) and course cards inside. Use **dnd-kit** for drag and drop. When a card is dropped, call the API to save the new semester, then refetch.
5. **Validation display in the roadmap**: if a course is placed before its prerequisites, show a red warning on the card. The backend decides what is valid (never trust only the frontend); the frontend just displays it.

Step 5.4: Habits that save time

- Handle three states on every data-driven component: **loading**, **error**, **empty**.
- Keep server data in TanStack Query, not in `useState`. Use `useState` only for local UI things like whether a panel is open.
- Start with ugly and working. Polish after the Roadmap works.

Checkpoint (end of Phase 4): a user can search courses, pick a major, mark completed courses, see progress, and drag courses into semesters, all on seeded data.

Part 6: The scraper

Now that you know exactly what shape the data must take, build the scraper.

Step 6.1: Investigate before coding

1. Open GMU's online course catalog in your browser and find where courses are listed.
2. Check `robots.txt` and the site's terms of use for what automated access is allowed. If there is an official export, data feed, or API, **use that instead of scraping HTML**.
3. Right-click a course entry and choose **Inspect**. Find the HTML elements that hold the course code, title, credits, description, and prerequisites. Note the CSS selectors.

4. Check whether the page content is in the raw HTML or loaded by JavaScript after the page opens. Jsoup only sees raw HTML. If the data appears only after JavaScript runs, you will need to find the underlying data request in the browser's Network tab, or use a headless-browser tool instead.
5. Save two or three example pages as files in your repo under `docs/samples/`. You will write the parser against these files.

Step 6.2: Add Jsoup

Add to `backend/pom.xml` (check the latest version on Maven Central):

```
<dependency>
  <groupId>org.jsoup</groupId>
  <artifactId>jsoup</artifactId>
  <version>LATEST_VERSION</version>
</dependency>
```

Step 6.3: Structure the scraper as separate stages

This is the key design decision. Do **not** write one giant method that fetches, parses, and saves.

```
Fetch   -> download a page, save the raw HTML to scraper-cache/ (be polite:
delay between requests)
Parse   -> HTML -> ParsedCourse objects (no database here)
Validate-> check required fields, flag problems
Save    -> upsert into the course table
```

Benefits: you fetch each page once (the cache), parser bugs can be fixed and re-run without hitting the website again, and the parser can be unit tested against your saved sample pages.

Define an interface so other universities can plug in later:

```
public interface CatalogScraper {
    String universitySlug();
    List<ParsedCourse> scrape();
}
```

`GmuCatalogScraper` is the first implementation.

Step 6.4: Fetching and parsing with Jsoup

```
Document doc = Jsoup.connect(url)
        .userAgent("DegreePlanner student project (contact: your-email)")
        .timeout(15000)
        .get();

for (Element block : doc.select("SELECTOR-YOU-FOUND-IN-INSPECT")) {
    String heading = block.select("SELECTOR-FOR-TITLE").text();
    // heading often looks like 'CS 211 Object-Oriented Programming'
    Matcher m = Pattern.compile("^([A-Z]{2,4})\\s+(\\d{3}[A-Z]?)\\s+
(.*)$").matcher(heading);
    if (!m.find()) { log.warn("Could not parse heading: {}", heading);
continue; }
    // subject = m.group(1), number = m.group(2), title = m.group(3)
}
```

The selectors are placeholders: yours come from Step 6.1. Real catalogs differ per page type, so expect to adjust.

Step 6.5: Parse credits

Credits appear as `3`, `1-3`, or `3 credits`. Extract numbers with a regex, set `credits_min` and `credits_max`. If you cannot find any, record the course as a validation failure rather than guessing 3.

Step 6.6: Parse prerequisites (the hard part)

Work in layers and stop when the results are good enough:

1. **Extract the prerequisite sentence** from the description (for example text after `Prerequisite(s):` up to the period).
2. **Find all course codes** with a regex. If there are none ("permission of instructor"), set `parse_status = NEEDS_REVIEW`.
3. **Handle simple structures**: codes joined by `and`, `or`, commas, and parentheses. Tokenize into codes, `AND`, `OR`, `(`, `)`, then build the tree with a small recursive-descent parser. `AND` binds tighter than `OR`.
4. **Ignore or flag** grade requirements ("with a C or better"), placement scores, and class standing. Keep the raw text; the UI shows it.

5. **Print a report**: how many courses parsed cleanly, how many are flagged. Read 30 flagged ones, look for the most common patterns, and fix those. Stop around 80 to 90 percent clean. The rest can be fixed by hand with a small admin script or SQL.

Corequisites follow the same logic and go in `coreq_text`.

Step 6.7: Validate and save

- Required: subject, number, title, credits. Missing any -> log and skip the record, and count it.
- **Upsert** (insert if new, update if existing) by `(university_id, subject, number)` so re-running the scraper never creates duplicates.
- Set `scraped_at` so you know how fresh data is.
- Run the scraper from a command-line trigger or an admin-only endpoint, **not** automatically on every startup.

Step 6.8: Degree requirements and offerings

Program pages are harder to parse than courses because the structure varies. Pragmatic approach: scrape what is easy (the list of courses on a program page), then enter group structure and rules **manually** into a seed file for each supported major. For a portfolio project, fully supporting 3 to 5 majors accurately beats poorly supporting 100. Fill `course_offering` from the schedule of classes (Fall/Spring/Summer) or manually for core courses.

Checkpoint (end of Phase 2): the course table holds a few thousand real courses, a log shows how many failed validation, and rerunning does not duplicate anything.

Part 7: Accounts and login

Before this point you used a fixed test user. Now add real users.

1. Add the **Spring Security** dependency, plus a JWT library (for example `jjwt`).
2. `POST /api/auth/register`: validate the email, hash the password with **BCrypt** (never store plain passwords), save the user.
3. `POST /api/auth/login`: verify the password, return a signed **JWT** with an expiry.
4. A security filter reads the `Authorization: Bearer <token>` header, validates it, and identifies the user. Make `/api/auth/**`, `/api/courses/**` and `/api/programs/**` public and everything under `/api/me/**` authenticated.

5. Replace the hardcoded test user: controllers take the current user from the security context.
6. **Frontend:** a login/register page, store the token, and add an axios interceptor that attaches it to every request. Storing in `localStorage` is simple and acceptable for a student project; note in the README that production apps often use secure cookies instead. Redirect to login on a 401 response.
7. Keep the signing secret in an environment variable, never in Git.

If you want a quicker path, you could use a third-party login (Google OAuth, or a service like Auth0/Clerk). The trade-off is less to build but more setup and dependence on an outside service.

Checkpoint: two accounts see different completed courses.

Part 8: Automatic degree plan generation

This is the most interesting part. Build it as a plain Java class (`planner/PlanGenerator`) with **no database or web code inside**: it takes data in and returns a plan. That makes it easy to test.

Inputs

- Courses the student still needs (from the requirement tree; for CHOOSE_N groups, pick the courses to take, starting with the simplest rule: the first N valid ones, then add preferences)
- Completed courses
- Prerequisite trees and corequisites
- Offerings per course (Fall, Spring, Summer)
- Settings: starting term, min/max credits per semester (for example 12 to 18), include summers or not

Algorithm (greedy, layered)

1. **Build a dependency graph.** An edge runs from each prerequisite course to the courses that depend on it. Only include prerequisites that are still needed.
2. **Detect problems first.** A cycle in the graph, or a prerequisite that is not offered and not completed, means the plan is impossible: return a clear error naming the courses.

3. **Compute priority.** For each course, count the longest chain of courses that depend on it (its *depth*). Courses with long chains must start early (the critical path).
4. **Loop over semesters** starting from the student's first term:
   - Build the list of **eligible** courses: all prerequisites completed in *earlier* semesters (or corequisites in this one), and offered in this term type.
   - Sort eligible courses by priority (deepest chain first).
   - Add courses until the credit cap is reached.
   - Mark them as taken and move to the next semester.
5. **Stop** when all required courses are placed, or after a sanity limit (for example 12 semesters), in which case report that it cannot be completed.
6. **Fill to the minimum.** If a semester is under the minimum credits and there are no more required courses available, add electives or leave it light and show a note.

Alternatives

Once the base works, generate variations by changing settings and re-running: light semesters (max 12 credits), fast track (max 18, use summers), or "hardest courses early" versus "balanced difficulty." Each variation is just a different sort order or cap, not new code paths.

API and UI

- `POST /api/plans/generate` with the settings returns a list of semesters with courses and credit totals, plus any warnings.
- The frontend shows it in the Roadmap page, and the student can then drag courses to adjust. Every move is re-checked by the same prerequisite evaluator.

Checkpoint: for your seeded major, clicking Generate fills a valid plan, and a deliberately impossible setup (a prerequisite cycle in test data) produces a clear error.

Part 9: Testing

Write tests as you go, not at the end.

- **Unit tests (JUnit 5)**: the prerequisite evaluator, the prerequisite parser (use real text examples from the catalog), the progress calculator, and the plan generator. These are your most valuable tests because they are pure logic.

- **Parser tests**: save sample HTML in `src/test/resources` and assert that the scraper extracts the right code, title, and credits.
- **Integration tests (Spring Boot Test + Testcontainers)**: start a real PostgreSQL in Docker during tests, run Flyway, and call the API with MockMvc.
- **Frontend tests (Vitest + React Testing Library)**: a few tests for key components, such as the course card and progress bar.
- **CI with GitHub Actions**: a workflow that runs `mvn test` in `backend/` and `npm test` plus `npm run build` in `frontend/` on every pull request. Add a status badge to the README.
- **Scraper failure handling**: if the site layout changes, validation counts will spike; make the scraper exit with a clear failure message when more than some threshold (say 20 percent) of records fail, instead of loading garbage into the database.

Part 10: Deployment

Hosting services and free tiers change, so check current pricing and limits before committing. A typical setup:

1. **Database**: a managed PostgreSQL (for example Neon, Supabase, Railway, or Render). Create a database and copy its connection string.
2. **Backend**: write a `Dockerfile` for the Spring Boot app, then deploy it to a service such as Render, Railway, or Fly.io. Set environment variables there: database URL, username, password, JWT secret, and the allowed frontend origin.
3. **Make config environment-driven.** In `application.yml`, use `${DB_URL}`-style placeholders with local defaults, and make the CORS allowed origin come from a variable too.
4. **Frontend**: deploy `frontend/` to Vercel, Netlify, or Cloudflare Pages. Set `VITE_API_URL` to your deployed backend URL. Because this is a single-page app, enable the rewrite-all-to-index.html setting so page refreshes on `/roadmap` work.
5. **Run migrations**: Flyway runs automatically on backend startup. Run the scraper (or load a seed dump) once against the production database.
6. **Smoke test** the live site end to end: register, pick a major, generate a plan.
7. **README**: finish it with a project description, screenshots, architecture diagram, tech stack, local setup steps (clone, `docker compose up`, run backend, run frontend), how to run the scraper, how to run tests, and a live demo link. This is what recruiters read first.

Part 11: Timeline and team workflow

A realistic plan for a part-time student team (adjust to your pace):

Weeks	Work
1	Tools installed, skeleton running, database created (Parts 1 to 3)
2 to 3	Seed one major, build course and program APIs, prerequisite evaluator (Part 4)
3 to 5	Frontend catalog, profile, dashboard (Part 5)
5 to 7	Roadmap with drag and drop (Part 5)
6 to 9	Scraper and prerequisite parser, in parallel with the frontend (Part 6)
9 to 10	Login (Part 7)
10 to 13	Plan generator and alternatives (Part 8)
13 to 15	Tests, CI, deploy, README (Parts 9 and 10)

Dividing work among teammates

Split by area so people are rarely editing the same files:

- **Backend core**: entities, repositories, progress calculation, auth
- **Data**: scraper, prerequisite parser, seeding majors
- **Frontend**: pages, components, drag and drop
- **Planner / DevOps**: plan generator, tests, CI, deployment

Rotate roles between phases so everyone learns every layer. Agree on **API contracts first**: before the backend and frontend each build a feature, write down the endpoint, the request, and the example JSON response in `docs/api.md`. Then the frontend can build against a fake response while the backend is still in progress.

When you get stuck

- Read the **first** error in the stack trace and the line mentioning your own package.

- Shrink the problem: test the endpoint in Postman before blaming the frontend; query the database in DBeaver before blaming the backend.
- Commit often. A working commit is a safe place to return to.
- Keep a `docs/decisions.md` where you write down choices and why (for example how you handle double counting). It will help in interviews.

Good future extensions

More majors and universities (the `CatalogScraper` interface and `university_id` are already prepared), minors and double majors, transfer and AP credit, a PDF export of the plan, and GPA or workload estimates per semester.